

PROJECT BASED EXPERIMENTAL LEARNING

COURSE – NO SQL DATABASE

PROJECT REPORT

TOPIC – MONGODB CRUD OPERATIONS

SUBMITTED TO	Mr. Harendra Singh
SUBMITTED BY	Devansh Singh
ROLL NO.	1250258183
CLASS	BCA - DS23
SUBJECT CODE	BCADSN23202
UNIVERSITY	Babu Banarasi Das University, Lucknow

ACKNOWLEDGMENT

I would like to express my sincere thanks to my mentor, Mr. Harendra Singh, for his guidance and encouragement throughout this project. His support helped me understand how MongoDB works in practice and how the four CRUD operations – Create, Read, Update and Delete – are applied to a real dataset.

I am also grateful to Babu Banarasi Das University, Lucknow, for providing the academic environment, laboratory facilities and resources needed to complete this practical work.

ABSTRACT

This project demonstrates CRUD operations using MongoDB, a NoSQL, document-oriented database. A database named **company** and a collection named **employees** were created, and employee records were inserted using `insertMany()` and `insertOne()`. The data was then retrieved with `find()` and `findOne()` using filters, comparison operators, the `$in` operator, projection, sorting and limiting.

Records were modified with `updateOne()` and `updateMany()` using the `$set` and `$inc` operators, and removed with `deleteOne()` and `deleteMany()`. All work was carried out in the MongoDB Shell (mongosh), and the output was verified after every major step.

Keywords: MongoDB, NoSQL, CRUD, MongoDB Shell, Employees Collection, Documents

INTRODUCTION

MongoDB is a NoSQL, document-oriented database that stores data as flexible, JSON-like documents in a binary format called BSON. Unlike a relational database, it does not force every record into a fixed table schema, so documents in the same collection can have different fields. This makes it well suited to data that changes shape over time.

CRUD stands for the four basic operations every database must support: **Create** (insert new data), **Read** (retrieve data), **Update** (modify existing data) and **Delete** (remove data).

In this project a **company** database with an **employees** collection was used. Each document stores an employee ID, name, department, years of experience, salary and city. The practical covers insertion, retrieval, filtering, sorting, updating and deletion of these records.

The collection starts with 22 employee documents. After the update and delete operations are complete, 19 documents remain.

METHODOLOGY

The project followed this workflow:

- Use MongoDB Community Server, MongoDB Shell (mongosh) and MongoDB Compass as the working tools.
- Create the **company** database and the **employees** collection.
- Insert 21 documents with `insertMany()` and one more with `insertOne()`, giving 22 in total.
- Read records with `find()` and `findOne()`, using conditions and comparison operators.
- Apply multiple conditions, the `$in` operator, projection, sorting and limiting.
- Update records with `updateOne()`, `$set`, `$inc` and `updateMany()`.
- Delete records with `deleteOne()` and `deleteMany()`.
- Verify the final collection with `find()` and `countDocuments()`.

Tools Used

Tool	Purpose
MongoDB Community Server	Runs the database server
MongoDB Shell (mongosh)	Command-line interface used to run all queries
MongoDB Compass	GUI for viewing databases and collections

IMPLEMENTATION DETAILS

All commands below were run in mongosh on the **employees** collection.

1. Create database and collection

```
use company
db.createCollection("employees")
```

Output: switched to db company; { ok: 1 }. The database and collection were created successfully.

2. Create operation – insert multiple documents

```
db.employees.insertMany([
  { emp_id: 1, name: "Devansh Singh", department: "IT",
    experience: 5, salary: 62000, city: "Lucknow" },
  { emp_id: 2, name: "Rohit Malhotra", department: "Sales",
    experience: 3, salary: 48000, city: "Kanpur" },
  { emp_id: 3, name: "Sanya Kapoor", department: "IT",
    experience: 6, salary: 71000, city: "Lucknow" },
  ...
])
```

21 documents were inserted with insertMany(). The full dataset contains employees from the IT, Sales, HR, Finance and Marketing departments.

3. Create operation – insert one document and count

```
db.employees.insertOne(
  { emp_id: 21, name: "Amitabh Sinha", department: "Sales",
    experience: 2, salary: 37000, city: "Prayagraj" }
)
db.employees.countDocuments()
```

Output: { acknowledged: true, insertedId: ObjectId('...') } and 22. The collection now holds 22 documents.

4. Read operation – display and search

```
db.employees.find().pretty()
db.employees.findOne({ name: "Sanya Kapoor" })
db.employees.find({ department: "IT" })
```

The first query displayed all 22 documents, the second returned the single record of Sanya Kapoor, and the third returned the 7 employees of the IT department.

IMPLEMENTATION DETAILS (CONTINUED)

5. Query operators

```
db.employees.find({ salary: { $gt: 60000 } })
db.employees.find({ department: "IT", city: "Lucknow" })
db.employees.find({ department: { $in: ["Sales", "HR"] } })
```

These queries demonstrate a comparison operator (8 employees earning more than 60,000), multiple conditions (6 IT employees in Lucknow) and the \$in operator (11 employees in Sales or HR).

6. Projection, sorting and limiting

```
db.employees.find({}, { name: 1, salary: 1, _id: 0 })
  .sort({ salary: -1 })
  .limit(5)
```

Only name and salary were selected, sorted by salary in descending order, and limited to five documents – the top five earners.

7. Update operation

```
db.employees.updateOne(
  { name: "Devansh Singh" },
  { $set: { salary: 65000 } }
)
db.employees.updateOne(
  { name: "Vikram Rathore" },
  { $inc: { salary: 3000 } }
)
```

Devansh Singh's salary changed from 62000 to 65000 using \$set. Vikram Rathore's salary rose from 39000 to 42000 using \$inc.

8. Update multiple documents

```
db.employees.updateMany(
  { city: { $ne: "Lucknow" } },
  { $set: { remote: true } }
)
db.employees.updateMany(
  { department: "Sales", experience: { $lt: 3 } },
  { $set: { experience: 3 } }
)
```

A new field remote: true was added to the 15 employees living outside Lucknow. Four Sales employees with less than 3 years of experience were updated to 3 years.

IMPLEMENTATION DETAILS (CONTINUED)

9. Delete operation

```
db.employees.deleteOne({ name: "Suraj Pillai" })
db.employees.deleteMany({ salary: { $lt: 40000 } })
```

One record (Suraj Pillai) was deleted with `deleteOne()`. Two employees with a salary below 40,000 (Aakash Dixit and Amitabh Sinha) were deleted with `deleteMany()`.

10. Verify the final collection

```
db.employees.find().pretty()
db.employees.countDocuments()
```

The final check confirmed that 19 documents remain in the collection.

Summary of operations

Step	Operation	Documents affected
Create	<code>insertMany()</code>	21 inserted
Create	<code>insertOne()</code>	1 inserted (total 22)
Update	<code>updateOne()</code> with <code>\$set</code>	1 modified
Update	<code>updateOne()</code> with <code>\$inc</code>	1 modified
Update	<code>updateMany()</code> – add remote field	15 modified
Update	<code>updateMany()</code> – Sales experience	4 modified
Delete	<code>deleteOne()</code>	1 deleted
Delete	<code>deleteMany()</code>	2 deleted (total 19)

RESULTS

All CRUD operations were performed successfully on the employees collection, and the output was verified after insertion, searching, filtering, updating and deletion.

The top five earners before the update and delete stage were:

Name	Salary
Neelam Rana	91000
Manish Tandon	88000
Rajeev Nair	82000
Karan Mehta	78000
Tanya Arora	75000

Number of documents at each stage:

Stage	Documents in collection
After insertMany()	21
After insertOne()	22
After updates	22
After deleteOne()	21
After deleteMany()	19

Final CRUD verification:

- **Create:** Employee documents were inserted successfully.
- **Read:** Documents were retrieved with find(), findOne() and filtered queries.
- **Update:** Single and multiple documents were modified successfully.
- **Delete:** Single and multiple documents were deleted successfully.

Final collection (19 documents)

Emp ID	Name	Dept	Exp	Salary	City	Remote
1	Devansh Singh	IT	5	65000	Lucknow	-
2	Rohit Malhotra	Sales	3	48000	Kanpur	true
3	Sanya Kapoor	IT	6	71000	Lucknow	-
4	Vikram Rathore	Sales	3	42000	Varanasi	true
5	Ishita Bhatt	HR	4	52000	Prayagraj	true
6	Karan Mehta	IT	7	78000	Lucknow	-
7	Priyanka Joshi	Sales	3	45000	Kanpur	true
8	Nikhil Saxena	IT	2	41000	Lucknow	-
9	Meenal Trivedi	HR	5	58000	Agra	true
11	Rajeev Nair	Finance	8	82000	Gorakhpur	true
12	Tanya Arora	IT	6	75000	Lucknow	-
14	Alka Bhargava	HR	3	49000	Kanpur	true
15	Manish Tandon	Finance	9	88000	Varanasi	true
16	Ritika Chandra	IT	4	60000	Bareilly	true
17	Deepak Rawat	Sales	3	42000	Lucknow	-
18	Swati Agnihotri	HR	6	63000	Agra	true
19	Gaurav Khanna	Marketing	5	57000	Noida	true
20	Neelam Rana	IT	8	91000	Lucknow	-
22	Pooja Lakhani	Marketing	4	54000	Kanpur	true

CONCLUSION

This project shows how MongoDB can be used to manage employee data through the four CRUD operations. Records were created, read with a range of filters and operators, updated individually and in bulk, and deleted selectively, with the result checked at every step.

The practical gave a clear understanding of how document-based databases work and how flexible they are – for example, the remote field could be added to some documents without changing any schema. The final verification confirmed that all operations ran correctly and that 19 documents remained in the collection.

FUTURE SCOPE

- Build a complete Employee Management System with a web or desktop interface.
- Add authentication and role-based access for administrators and employees.
- Add more collections and fields such as attendance, leave, payroll and projects.
- Connect MongoDB to Python, Java or a web application for real-time data handling.
- Add reporting, search and analytics features, such as department-wise salary summaries.

REFERENCES

- MongoDB documentation and MongoDB Shell concepts used for CRUD operations.
- MongoDB Community Server / mongosh practical environment.
- MongoDB Compass for database and collection management.

THANK YOU!